

//new & delete operator example 1

```
#include <iostream>
using namespace std;

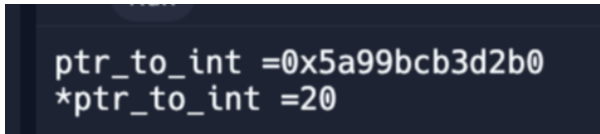
int main() {

    //int x = 10;

    int *ptr_to_int = new int(20);
    //*ptr_to_int = 20;

    cout << "ptr_to_int =" << ptr_to_int << endl;
    cout << "*ptr_to_int =" << *ptr_to_int << endl;

    return 0;
}
```



```
ptr_to_int =0x5a99bcb3d2b0
*ptr_to_int =20
```

//new and delete operator example 2

```
#include <iostream>
using namespace std;

int main() {

    int x = 10;
    int *ptr_to_int = new int(20);

    cout << "ptr_to_int =" << ptr_to_int << endl;
    cout << "*ptr_to_int =" << *ptr_to_int << endl;

    ptr_to_int = new int (100);

    delete ptr_to_int;

    *array[0] = 1;

    return 0;
}
```

```
./main.cpp:16:6: error: use of class template 'array' requires template arguments
    16 |     *array[0] = 1;
        |     ^
/nix/store/14c6s4xzh14i2b05s00rjns2j93gzz4-gcc-13.2.0/include/c++/13.2.0/tuple:2005:45: note: template is declared here
    2005 |     template<typename _Tp, size_t _Nm> struct array;
        |     ^
1 error generated.
make: *** [Makefile:10: main] Error 1
```

//new and delete operator example 3

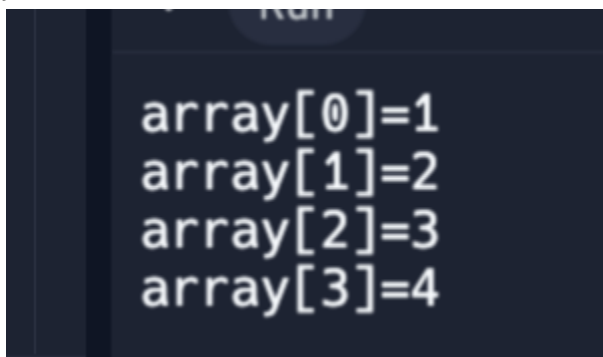
```
#include <iostream>
using namespace std;
int main(){

    double* array = new double[4];

    array[0] = 1;
    array[1] = 2;
    array[2] = 3;
    array[3] = 4;

    for (int i = 0; i < 4; i++)
        cout << "array[" << i << "]= " << array[i] << endl;

    return 0;
}
```



```
array[0]=1
array[1]=2
array[2]=3
array[3]=4
```

//new and delete operator example 4

```
#include <iostream>
using namespace std;
int main(){
```

```

double* array = new double[4];

array[0] = 1;
array[1] = 2;
array[3] = 3;
array[4] = 4;

for (int i =0; i < 4; i++){
    cout << "array[" << i << "]=" << array[i] << endl;

delete[] array;

}
}

```

```

array[0]=1
array[1]=3.62568e+67
free(): double free detected in tcache 2

```

//Accessing methods using a string pointer

```

#include <iostream>
#include <string>
using namespace std;

int main() {

    string* strPtr = new string("Hello World");

    cout << "Length of the string: " << strPtr->length() << endl;

    delete strPtr;
    return 0;

}

```

```

Length of the string: 11

```

//Accessing methods using a string pointer 2

```

#include <iostream>
#include <string>
using namespace std;

int main() {

```

```

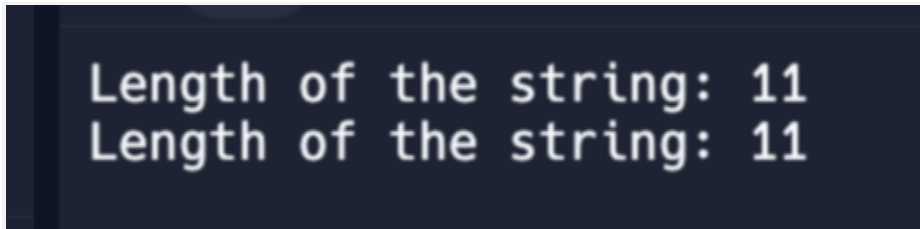
string* strPtr = new string("Hello World");

cout << "Length of the string: " << strPtr->length() << endl;
cout << "Length of the string: " << (*strPtr).length() << endl;

delete strPtr;

return 0;
}

```



```

Length of the string: 11
Length of the string: 11

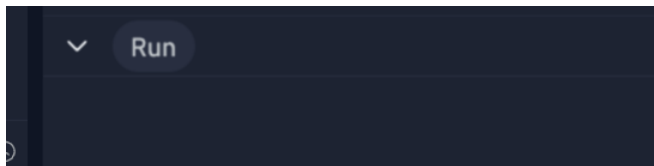
```

//Rectangle class definition (public vs private)

```

#include <iostream>
using namespace std;
int main(){
class Rectangle {
    public:
        Rectangle (int a, int b): width(a), height(b) {}
        ~Rectangle() {
            cout << "Destructor called" << endl;
        }
        int area () {
            return (width*height);
        }
    private:
        int width, height;
};
}

```



//Full rectangle code

```

#include <iostream>
using namespace std;

class Rectangle {

```

```

public:
    Rectangle (int a, int b): width(a), height(b) {}
    ~Rectangle() {
        cout << "Destructor called" << endl;
    }
    int area () {
        return (width*height);
    }
private:
    int width, height;
};

int main() {

    Rectangle rect1(5, 5);

    Rectangle* rect2 = new Rectangle(3, 4);

    cout << "Area of rect1: " << rect1.area() << endl;

    cout << "Area of rect2 (using dereference): " <<
    (*rect2).area() << endl;

    cout << "Area of rect2 (using arrow operator): " <<
    rect2->area() << endl;

    delete rect2;
    return 0;
}

```

```

Area of rect1: 25
Area of rect2 (using dereference): 12
Area of rect2 (using arrow operator): 12
Destructor called
Destructor called

```